Semántica

Juan Manuel Rabasedas

Semántica de un lenguaje

¿Cómo saber si dos programas son equivalentes?

```
int add1 (int x, int y){ return (x+y);}

int add2 (int x, int y){ return (x+y);}
```

Semántica de un lenguaje

- (if b then p else q); r = if b then (p;r) else (q;r)
- p; if b then q else r = if b then (p;q) else (p;r)
- p; if b then q else r = if (p;b) then q else r
- b/0 = Null

Semántica Formal

- Para saber si dos programas son equivalentes:
  - Dar una descripción precisa de los programas.
  - Definir qué es lo que se puede observar de un programa.
  - Elegir una noción apropiada de equivalencia

Beneficios de semántica formal

- Implementación: compiladores, optimizaciones y análisis estático
- Verificación: soporte para razonar acerca de las propiedades de los programas.
- Diseño de Lenguajes: Resolver interacciones entre las características del lenguaje.

Enfoques para Semántica

Tenemos varios enfoques para dar la semántica de un lenguaje:

- **Operacional:** El significado de un programa está dado por los pasos de computación que el programa realiza.
- **Denotacional:** Dar un objeto matemático D y definir una función de interpretación

  $$[\,-\,] : T \to D$$

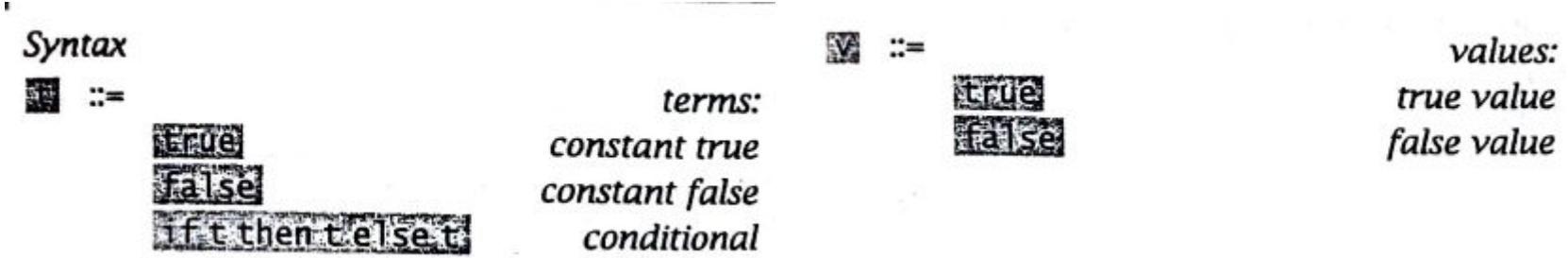
- **Axiomática:** Dar leyes sobre los programas. El significado es todo lo que se puede derivar de esas leyes. Se utiliza en los lenguajes imperativos (Ej: Hoare Logic)

Semántica Operacional

- Definir los pasos que un programa da durante su evaluación
- Las propiedades surgen del análisis de esta ejecución.
  - small step : la evaluación de las expresiones se hace pasos por paso.
  - big step: los pasos intermedios se ignoran y se da directamente el resultado.

Valores en Semántica Operacional

- Tomamos el fragmento booleano del lenguaje visto y los términos:



Relación de Evaluación big step

$$v \Downarrow v \qquad \text{(B-VALUE)}$$

$$\frac{t_1 \Downarrow \text{true} \qquad t_2 \Downarrow v_2}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \Downarrow v_2} \qquad \text{(B-IFTRUE)}$$

$$\frac{t_1 \Downarrow \text{false} \qquad t_3 \Downarrow v_3}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \Downarrow v_3} \qquad \text{(B-IFFALSE)}$$

- Definimos la relación $\Downarrow \subseteq T \times V$
- $t \Downarrow v$ se lee “t evalúa al valor v”
- La relación $\Downarrow$ es la menor relación que satisface las 3 reglas

Las reglas son esquemas

- En las reglas t_1, t_2,t_3 y v son **metavariables**.
- Cada regla es un esquema para una cantidad (posiblemente infinita) de reglas como:

$$\frac{}{T \Downarrow T}$$

$$\frac{T \Downarrow F \qquad F \Downarrow F}{\text{if } T \text{ then } T \text{ else } F \Downarrow F}$$

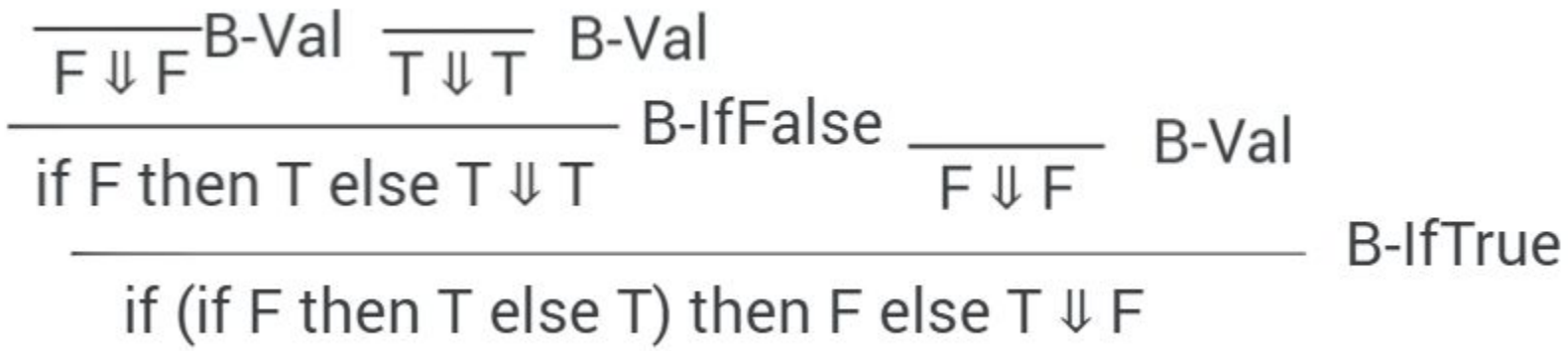
Relación de evaluación (Formalmente)

- Un **instancia** de una regla de inferencia se obtiene reemplazando consistentemente cada metavariable por el mismo término en la conclusión como en las premisas.
- Una regla satisface una relación, si cada instancia de la regla:
  - La conclusión está en la relación, o bien,
  - Alguna de las premisas no está en la relación.
- La relación de evaluación $\Downarrow$ es la menor relación binaria sobre términos que satisface las reglas.
- Cuando (t,v) decir que **el juicio de evaluación** $t \Downarrow v$ es **derivable**

Derivabilidad

- Qué $\Downarrow$ sea la menor relación nos dice que $t \Downarrow v$ es derivable si y sólo si sigue de las reglas:
  - es el axioma B-Val,
  - o bien es la conclusión de B-IfTrue o B-IfFalse con una premisa derivable.
- La derivabilidad de un juicio se justifica con un árbol de derivación:
  - Las hojas del árbol son etiquetas con instancias de B-Val
  - Los nodos internos son etiquetas con instancias de B-IfTrue o de B-IfFalse

Ejemplo



Ejercicio:
if F then T else (if T then F else T) ⇓ F

Relación de evaluación de small-step

- La semántica se da por una relación entre estados de una máquina abstracta.
- Definimos la relación de evaluación $\longrightarrow \subseteq \mathcal{T} \times \mathcal{T}$

$$\text{if T then } t_2 \text{ else } t_3 \longrightarrow t_2 \quad \text{(E-IfTrue)}$$

$$\text{if F then } t_2 \text{ else } t_3 \longrightarrow t_3 \quad \text{(E-IfFalse)}$$

$$\frac{t_1 \longrightarrow t_1'}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \longrightarrow \text{if } t_1' \text{ then } t_2 \text{ else } t_3} \quad \text{(E-If)}$$

- $t \longrightarrow t'$ se lee “t evalúa a t’ en un solo paso

Acerca de la relación de Evaluacion small step

- Notar que T y F no evalúan a nada
- Las reglas se dividen en reglas de computación (E-IfTrue E-IfFalse) y reglas de congruencia (E-If)
- La relación de evaluación fija una estrategia de evaluación:

  if t_1 then t_2 else t_3, se debe evaluar t_1 antes de evaluar t_2 o t_3

Ejercicio

$$\text{if T then } t_2 \text{ else } t_3 \longrightarrow t_2 \quad \text{(E-IfTrue)}$$
$$\text{if F then } t_2 \text{ else } t_3 \longrightarrow t_3 \quad \text{(E-IfTrue)}$$

$$\frac{t_1 \longrightarrow t_1'}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \longrightarrow \text{if } t_1' \text{ then } t_2 \text{ else } t_3} \quad \text{(E-If)}$$

$$\frac{t_2 \longrightarrow t_2'}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \longrightarrow \text{if } t_1 \text{ then } t_2' \text{ else } t_3}$$

$$\frac{t_3 \longrightarrow t_3'}{\text{if } t_1 \text{ then v else } t_3 \longrightarrow \text{if } t_1 \text{ then v else } t_3'}$$

$$\frac{t_1 \longrightarrow t_1'}{\text{if } t_1 \text{ then } v_2 \text{ else } v_3 \longrightarrow \text{if } t_1' \text{ then } v_2 \text{ else } v_3}$$

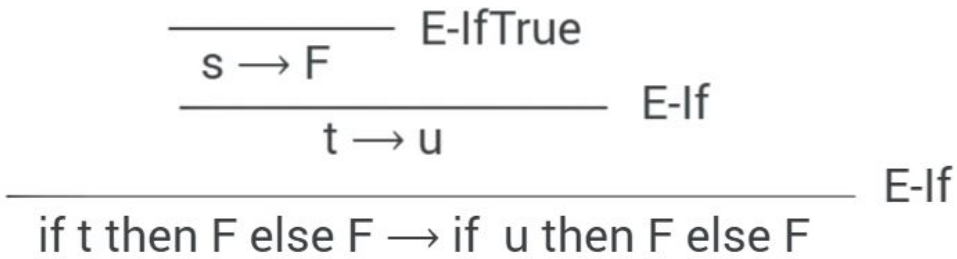
$$\text{if T then } v_2 \text{ else } v_3 \longrightarrow v_2$$
$$\text{if F then } v_2 \text{ else } v_3 \longrightarrow v_3$$

Ejemplo de árbol de derivación

s = if T then F else T
t = if s then T else T
u = if F then T else T

justificar

if t then F else F ⟶ if u then F else F



Inducción sobre una derivación

- Sea P un predicado sobre una derivación de juicios de evaluación.
- Si, para cada derivación D,
  - daso P(C), para todas las subderivaciones inmediatas C
  - podemos probar P(D)
- entonces P(D) vale para todo D

Determinismo de la evaluación small-step

Teorema:
si $t \longrightarrow t'$ y $t \longrightarrow t''$ entonces $t'=t''$

- Para probar el Teorema usaremos inducción sobre la derivación $t \longrightarrow t'$
- Si la última regla utilizada es E-IfTrue entonces
  t es de la forma if t_1 then t_2 else t_3 con $t_1 = T$
  Pero entonces:
  - la última regla no puede ser E-IfFalse ya que no podemos tener $t_1 = T$ y $t_1 = F$
  - Tampoco puede ser E-If ya que la regla pide que $t_1 \longrightarrow t'_1$ para algún t_1' pero $t_1 = T$ y T no deriva a ningún término.

Forma Normal

- Un termino esta en forma normal si no se le puede aplicar ninguna regla de evaluación.
- t está en forma normal si no existe t' tal que $t \longrightarrow t'$
- Para nuestro lenguaje las formas normales son T y F
- Teorema: Todo valor está en forma normal.
- En general el converso no vale (ej errores de ejecución)
- Para nuestro lenguaje tenemos el siguiente teorema:
  - Si t está en forma normal, entonces t es un valor

Evaluación de pasos múltiples

- la relación de pasos múltiples $\longrightarrow^*$ es la clausura reflexivo-transitiva de $\longrightarrow$
- Es decir la menor relación tal que:

$$\frac{t \longrightarrow t'}{t \longrightarrow^* t'}\ R1 \qquad \frac{}{t \longrightarrow^* t}\ R2 \qquad \frac{t \longrightarrow^* t' \quad t' \longrightarrow^* t''}{t \longrightarrow^* t''}\ R3$$

Teorema Unicidad de Forma Normal:
si $t \longrightarrow^* u$ y $t \longrightarrow^* u'$ entonces u y u' son formas normales entonces u=u'

Teorema (La evaluación Termina)
Para todo término t hay una forma norma t' tal que $t \longrightarrow^* t'$

Resultados

- Teorema Determinismo
  si t $\Downarrow$v y t $\Downarrow$ v' entonces v = v'
- Teorema de Terminación
  Para todo término t, existe v tal que t $\Downarrow$ v
- Teorema Equivalencia
  para todo termino t y valor v, t $\Downarrow$ v sii t $\rightarrow^*$ v

Semántica del Leng. de Exp Aritméticas

- Lenguaje de expresiones aritméticas:
  t :: T | F | **if** t **then** t **else** t | 0 | **succ** t | **pred** t | **iszero** t
- Para definir los valores agregamos una nueva categoría sintáctica de valores numéricos:
  v :: = T | F | nv
  nv ::= 0 | succ nv

Nuevas reglas de evaluación de paso chico

$$\frac{t_1 \to t_1'}{\texttt{succ}\ t_1 \to \texttt{succ}\ t_1'} \quad \text{(E-Succ)}$$

$$\texttt{pred}\ 0 \to 0 \quad \text{(E-PredZero)}$$

$$\texttt{pred}\ (\texttt{succ}\ nv_1) \to nv_1 \quad \text{(E-PredSucc)}$$

$$\frac{t_1 \to t_1'}{\texttt{pred}\ t_1 \to \texttt{pred}\ t_1'} \quad \text{(E-Pred)}$$

$$\texttt{iszero}\ 0 \to \texttt{T} \quad \text{(E-IsZeroZero)}$$

$$\texttt{iszero}\ (\texttt{succ}\ nv_1) \to \texttt{F} \quad \text{(E-IsZeroSucc)}$$

$$\frac{t_1 \to t_1'}{\texttt{iszero}\ t_1 \to \texttt{iszero}\ t_1'} \quad \text{(E-IsZero)}$$

- pred (succ (pred 0)) -> 0
- succ F **no es un valor** pero son formas normales
- Si t es una forma normal pero no es un valor, decimos que t está **atascado** (stuck)
- Un término atascado se puede pensar como error de run-time.

Semántica de un Lenguaje Imperativo

Sintaxis

intexp :: = nv | var | $-_u$ intexp | intexp + intexp | intexp $-_b$ intexp
| intexp **x** intexp | intexp % intexp

boolexp ::= true | false | intexp = intexp | intexp < intexp
| intexp > intexp | boolexp && boolexp
| boolexp **||** boolexp | **not** boolexp

comm ::= skip
| var := intexp
| comm ; comm
| if boolexp then comm else comm
| repeat comm boolexp

Representación de efectos laterales en la evaluación

- Este lenguaje tiene efectos laterales en la modificación del estado global
- Modelamos el estado global Σ como una función de identificadores en valores enteros:

$$\Sigma : var \to nv$$

- Las funciones de evaluación deben tener en cuenta el estado

$\Downarrow_{int}$ incluida (intexp x Σ) x nv

$\Downarrow_{bool}$ incluida (boolexp x Σ) x bv

$\rightsquigarrow$ incluida (comm x Σ) x (comm x Σ)

Reglas de expresiones enteras

$$\Downarrow_{int} \text{ incluida } (\text{intexp} \times \Sigma) \times nv$$

- Regla para valores:

  $(v,\sigma) \Downarrow_{int} \sigma\ v$

- Uso de estado

$$\frac{e0 \Downarrow int\ n0 \quad e1 \Downarrow int\ n1}{e0 + e1 \Downarrow int\ \ n0 + n1} \Rightarrow \frac{(e0,\sigma) \Downarrow int\ n0 \quad (e1,\sigma) \Downarrow int\ n1}{(e0 + e1,\sigma) \Downarrow int\ \ n0 + n1}$$

Reglas de expresiones booleanas

$$\Downarrow_{bool} \text{ incluida } (\text{boolexp} \times \Sigma) \times bv$$

- Similar al caso de expresiones enteras

$$\frac{(e0,\sigma) \Downarrow \text{int } n0 \quad (e1,\sigma) \Downarrow \text{int } n1}{(e0 = e1,\sigma) \Downarrow \text{bool } \; n0=n1}$$

Reglas de comandos

$$\rightsquigarrow \subseteq \text{incluida } (\text{comm} \times \Sigma) \times (\text{comm} \times \Sigma)$$

- Asignación

$$\frac{(e,\sigma) \Downarrow \text{int } n}{(v := e,\sigma) \rightsquigarrow (\text{skip},[\sigma \mid v : n])}$$

- Toda ejecución que termina lo hace en (skip,σ)
- Condicionales:

$$\frac{(e,\sigma) \Downarrow \text{bool true}}{(\text{if } e \text{ then } c0 \text{ else } c1, \sigma) \rightsquigarrow (c0,\sigma)} \qquad \frac{(e,\sigma) \Downarrow \text{bool false}}{(\text{if } e \text{ then } c0 \text{ else } c1, \sigma) \rightsquigarrow (c1,\sigma)}$$

Reglas de comandos

$\rightsquigarrow$ incluida (comm x Σ) x (comm x Σ)

- secuenciación

$$\frac{(c0\ ,\ \sigma) \rightsquigarrow (c0',\sigma')}{(c0\ ;\ c1,\ \sigma) \rightsquigarrow (c0';c1,\sigma')} \qquad \frac{}{(skip\ ;\ c1,\ \sigma) \rightsquigarrow (c1,\sigma)}$$

- Repeat

$$(\text{repeat c until b}, \sigma) \rightsquigarrow (c;\ \text{if b then skip else repeat x until b},\ \sigma)$$

Considerando errores

- Las expresiones enteras pueden fallar
- Agregamos un valor **err**$_i$

$$\Downarrow_{int} \subseteq (intexp \times \Sigma) \times (nv \cup \{\mathbf{err}_i\})$$

- Agregamos a las reglas:

$$\frac{(e1,\sigma) \Downarrow int\ 0}{(e0\ \%\ e1,\sigma) \Downarrow int\ \ erri}$$

- Reglas para expresiones booleanas:

$$\frac{(e0,\sigma) \Downarrow int\ erri}{(e0 = e1,\sigma) \Downarrow bool\ \ erri} \qquad \frac{(e0,\sigma) \Downarrow bool\ false\ \ (e1,\sigma) \Downarrow bool\ erri}{(e0\ \&\&\ e1,\sigma) \Downarrow bool\ \ erri}$$

$$\frac{(p,\sigma) \Downarrow bool\ erri}{(not\ p,\sigma) \Downarrow bool\ \ erri}$$

Considerando errores en comandos

$$\rightsquigarrow \subseteq (comm \times \Sigma) \times ((comm \cup \{\mathbf{err}_c\}) \times \Sigma)$$

$$\frac{(e,\sigma) \Downarrow \text{int erri}}{(v := e\ ;\ c1, \sigma) \rightsquigarrow (\text{errc}, \sigma)} \qquad \frac{(c0, \sigma) \rightsquigarrow \text{errc}}{(c0;c1, \sigma) \rightsquigarrow (\text{errc}, \sigma)}$$

$$\frac{(b,\sigma) \Downarrow \text{bool errb}}{(\text{repeat c until b}, \sigma) \rightsquigarrow (\text{errc}, \sigma)} \qquad \frac{(b,\sigma) \Downarrow \text{bool vb} \quad (c, \sigma) \rightsquigarrow \text{errc}}{(\text{repeat c until b}, \sigma) \rightsquigarrow (\text{errc}, \sigma)}$$

Agregamos E/S

- comm::= ....
  | input var
  | print var
- input v leer un entero (del teclado) y asigna el valor a la variable v
- print v imprime (en pantalla) el valor de v
- indicamos la entrada/salida etiquetando las transiciones
- la etiqueta n? indica una entrada n
- la etiqueta n! indica una salida n

$$\frac{}{(\text{input } v, \sigma) \overset{n?}{\rightsquigarrow} (\text{skip}, [\sigma \mid v{:}n])}$$

$$\frac{(e, \sigma) \Downarrow \text{int } n}{(\text{print } e, \sigma) \overset{n!}{\rightsquigarrow} (\text{skip}, \sigma)}$$

Propagación de errores E/S

$$\rightsquigarrow \subseteq (comm \times \Sigma) \times L \times ((comm \cup \{\mathbf{err}_c\}) \times \Sigma)$$

$$\frac{(c0, \sigma) \overset{a}{\rightsquigarrow} errc}{(c0;c1, \sigma) \overset{a}{\rightsquigarrow} (errc,\sigma)} \qquad \frac{(c0 , \sigma) \overset{a}{\rightsquigarrow} (c0',\sigma')}{(c0 ; c1, \sigma) \overset{a}{\rightsquigarrow} (c0';c1,\sigma')}$$

Donde a puede se nv? o un n! o **s** done **s** representa una transición silenciosa.
Donde $\overset{s}{\rightsquigarrow}$ es lo mismo que no etiquetar la transición $\rightsquigarrow$